

Methane & CO2 Tracker

Relatório da Fase 4 — Detecção de Anomalia (Isolation Forest)

Escopo desta fase: complementar a regra de alerta por limiar sustentado (Fase 2) com um classificador estatístico que sinaliza leituras fora do padrão histórico de cada sensor — incluindo casos que a regra de limiar não cobre, como picos isolados abaixo do threshold ou combinações incomuns entre concentração, temperatura e bateria.

Decisões de arquitetura desta fase

- Modelo **Isolation Forest** (scikit-learn), treinado sob demanda a cada leitura nova, isolado por par *sensor_id+gas_type* — cada sensor tem seu próprio perfil de normalidade, não um modelo global. Mesma filosofia sem estado em memória da regra de alerta (Fase 2): reavalia o histórico do banco a cada chamada, então o serviço pode reiniciar sem perder nenhum modelo treinado.
- Features: **concentration_ppm**, **temperature_c** e **battery_pct** — as três colunas já validadas e sempre presentes na ingestão (Fase 2). Combinar os três sinais permite detectar, por exemplo, uma leitura de concentração aparentemente normal acompanhada de uma temperatura incompatível com a operação normal do sensor.
- Guarda de *cold start*: um sensor novo, sem histórico suficiente (mínimo configurável via **ANOMALY_MIN_HISTORY**, padrão 50 leituras), nunca é classificado como anômalo por falta de dado — evita falsos positivos logo na instalação de um sensor.
- Histórico de treino limitado às leituras mais recentes (**ANOMALY_HISTORY_LIMIT**, padrão 500) para manter o custo de treino previsível e o modelo sensível a mudanças graduais de comportamento do sensor ao longo do tempo. Sensibilidade ajustável via **ANOMALY_CONTAMINATION** (padrão 2%).
- Nenhuma migração de schema foi necessária — a coluna *readings.is_anomaly* já existia desde a 0001_initial_schema, reservada para esta fase.
- Integração no pipeline de ingestão: chamado em *mqtt_consumer.py* logo após a avaliação da regra de alerta, marcando *Reading.is_anomaly* antes do commit final da leitura.

Arquivos entregues

Arquivo	Papel
backend/app/anomaly/detector.py	Classificador Isolation Forest por sensor+gás
backend/app/ingestion/mqtt_consumer.py	Chamada ao detector após cada leitura inserida
backend/app/db/session.py	Novas settings: anomaly_min_history/history_limit/contamination
.env.example / .env	Novas variáveis ANOMALY_*
backend/app/api/routers/readings.py	Novo filtro is_anomaly em GET /readings
frontend/src/components/ReadingsChart.tsx	Destaque visual (ponto vermelho) das leituras anômalas
frontend/src/index.css	Estilo da legenda do destaque de anomalia
backend/tests/test_anomaly_detection.py	Teste standalone — executado, OK

Status de verificação

- `test_anomaly_detection.py`: **executado com sucesso** — cobre três cenários: sensor sem histórico mínimo (não classifica), leitura dentro do padrão de operação normal (não marcada) e leitura claramente fora do padrão (marcada corretamente como anomalia).
- Suíte completa de testes standalone reexecutada após as mudanças (`test_api.py`, `test_mqtt_ingestion.py`, `test_alert_rule.py`, `test_alert_notifications.py`, `test_db_connection.py`, `test_proto_roundtrip.py`) — **nenhuma regressão** nos módulos de ingestão, alerta, notificação e API.
- Build de produção do dashboard (`tsc -b && vite build`) reexecutado com sucesso após o destaque visual de anomalias.
- scikit-learn precisou ser instalado no ambiente Python do host para rodar o teste standalone localmente — já constava em `requirements.txt`, mas não estava presente no ambiente do host (só usado dentro do container backend até então).

Não incluído nesta fase (próximos passos)

- Notificação automática ao detectar uma anomalia (hoje: só a flag `is_anomaly` é gravada, sem e-mail/webhook dedicado como o alerta de limiar tem).
- Geração de relatório PDF ANP/EPA do produto — módulo `app/reports` ainda vazio.
- Otimização de custo: o modelo é retreinado a cada leitura (aceitável no volume atual); cache de modelo ou retrain periódico ficam como melhoria futura se o volume de sensores crescer.

Gerado automaticamente por `backend/scripts/generate_phase4_report.py`